

ROVER NAVIGATION SIMULATOR

LINEAR ALGEBRA PROJECT --- DEVELOPMENT AND PREPARATION GUIDE

COURSE:	Linear Algebra
AUTHOR:	Sheikh Muhammad Muneeb
GROUP MEMBERS:	Sheikh M Muneeb Misha Jessani Adil Saleem
INSTRUCTOR:	Rameez Raghieb
SUBMISSION DATE:	30th July, 2026

Contents

1	PROJECT INTRODUCTION	1
1.1	WHAT IS THE ROVER NAVIGATION SIMULATOR	1
1.2	WHY LINEAR ALGEBRA MATTERS HERE	1
1.3	REAL-WORLD APPLICATIONS	1
2	PROJECT OBJECTIVES	3
2.1	TECHNICAL OBJECTIVES	3
2.2	MATHEMATICAL OBJECTIVES	3
3	MATHEMATICAL FOUNDATION	4
3.1	VECTORS AND ROVER STATE REPRESENTATION	4
3.2	COORDINATE SYSTEMS	4
3.3	TRANSLATION VECTORS	4
3.4	ROTATION MATRICES IN 2D	4
3.4.1	Optional: Rotation Matrices in 3D	5
3.5	HOMOGENEOUS COORDINATES	5
3.6	TRANSFORMATION MATRICES	5
3.7	MATRIX MULTIPLICATION AND TRANSFORMATION CHAINING	5
3.8	CONFIGURABLE INITIAL WORLD TRANSFORM	6
3.9	DERIVING A ROUTE BETWEEN TWO POINTS	6
4	PROPOSED SYSTEM ARCHITECTURE	8
4.1	PIPELINE OVERVIEW	8
4.2	COMPONENT DESCRIPTIONS	9
5	STEP-BY-STEP DEVELOPMENT PLAN	10
6	SUGGESTED IMPLEMENTATION OPTIONS	14
6.1	COMPARISON OF OPTIONS	14
6.2	RECOMMENDATION	14
7	ALGORITHM DESIGN	15
8	TESTING STRATEGY	19
8.1	TEST CASE CATEGORIES	20
8.2	TESTING CHECKLIST	21

9	PROJECT MANAGEMENT CHECKLIST	22
9.1	RESEARCH	22
9.2	IMPLEMENTATION	22
9.3	DOCUMENTATION	22
9.4	FINAL REVIEW	23
10	VIVA PREPARATION	24
10.1	VIVA READINESS CHECKLIST	24
10.2	MATHEMATICS QUESTIONS	24
10.3	IMPLEMENTATION QUESTIONS	25
10.4	PROJECT UNDERSTANDING QUESTIONS	26
11	PRESENTATION PREPARATION	27
11.1	RECOMMENDED 15-MINUTE STRUCTURE	27
12	FINAL PROJECT COMPLETION CHECKLIST	28

PROJECT INTRODUCTION

1.1 WHAT IS THE ROVER NAVIGATION SIMULATOR

The Rover Navigation Simulator models a planetary rover in an unknown terrain. The rover receives a sequence of movement commands (translate, rotate, or a combination of both) and the simulator computes the rover's resulting position and orientation after each command using linear algebra, specifically transformation matrices. The simulator tracks and visualizes the complete trajectory of the rover from its starting configuration to its final state.

1.2 WHY LINEAR ALGEBRA MATTERS HERE

CORE IDEA

Every movement of a rigid body in 2D or 3D space, whether a translation, a rotation, or a combination of the two, can be represented and computed using vectors and matrices. This project exists specifically to make that abstraction concrete.

- **Vectors** encode the rover's position and heading as numerical quantities that can be manipulated algebraically.
- **Matrices** encode linear transformations (rotation, translation) as operators acting on those vectors.
- **Matrix multiplication** allows multiple transformations to be composed into a single operation, which is essential when a rover executes a long sequence of commands.
- **Homogeneous coordinates** allow rotation and translation, which are mathematically different types of operations, to be unified into a single matrix multiplication.

1.3 REAL-WORLD APPLICATIONS

- **Planetary exploration:** Rovers such as NASA's Perseverance and Curiosity use transformation-matrix-based localization to track position on Mars.

- **Robotics:** Mobile robots and robotic arms rely on transformation chains to compute end-effector or chassis position relative to a base frame.
- **Autonomous navigation:** Self-driving vehicles use coordinate transformations to convert sensor data between local and global reference frames.
- **Computer graphics:** Game engines and rendering pipelines apply rotation, translation, and homogeneous coordinate machinery to orient objects in a 3D scene.

PROJECT OBJECTIVES

2.1 TECHNICAL OBJECTIVES

- ☐ Design and implement a rover state representation (position and orientation).
- ☐ Implement a command parser capable of interpreting movement instructions.
- ☐ Build a matrix operations engine supporting translation and rotation.
- ☐ Implement transformation chaining across an arbitrary sequence of commands.
- ☐ Store and visualize the rover's complete trajectory.
- ☐ Support a fully configurable starting pose and target end point at runtime, so the rover's mission is never hardcoded.
- ☐ Derive the trajectory connecting a start and end point automatically from displacement geometry, rather than requiring a hand-scripted command sequence.
- ☐ Produce a complete, human-readable matrix calculation trace for every transformation applied, suitable for direct instructor review.
- ☐ Validate correctness through structured test cases.

2.2 MATHEMATICAL OBJECTIVES

- ☐ Demonstrate correct construction of 2D (and optionally 3D) rotation matrices.
- ☐ Demonstrate correct construction of translation vectors and matrices.
- ☐ Demonstrate proper use of homogeneous coordinates to unify translation and rotation.
- ☐ Demonstrate correct order-dependent matrix multiplication for chained transformations.
- ☐ Verify numerical accuracy of trajectories against hand calculations.

MATHEMATICAL FOUNDATION

3.1 VECTORS AND ROVER STATE REPRESENTATION

The rover's state at any time step can be represented as a vector describing its position and heading:

$$\mathbf{s} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} \quad (3.1)$$

where x, y denote the rover's position in the 2D plane and θ denotes its heading angle relative to a fixed reference axis.

3.2 COORDINATE SYSTEMS

The simulator distinguishes between two frames of reference:

- **World frame:** A fixed global coordinate system in which the rover's absolute position is expressed.
- **Rover (local) frame:** A coordinate system attached to the rover itself, in which movement commands such as "move forward" are naturally expressed.

Every movement command issued in the rover's local frame must be transformed into the world frame before the rover's global state is updated.

3.3 TRANSLATION VECTORS

A pure translation shifts the rover's position without changing its heading:

$$\mathbf{p}' = \mathbf{p} + \mathbf{t}, \quad \mathbf{t} = \begin{bmatrix} t_x \\ t_y \end{bmatrix} \quad (3.2)$$

3.4 ROTATION MATRICES IN 2D

A rotation by angle θ about the origin is given by:

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \quad (3.3)$$

Applying this matrix to a position vector $\mathbf{p} = [x \ y]^T$ produces the rotated vector $\mathbf{p}' = R(\theta) \mathbf{p}$.

3.4.1 OPTIONAL: ROTATION MATRICES IN 3D

If the rover's motion is extended to three dimensions, rotation about the individual coordinate axes is given by:

$$R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (3.4)$$

with analogous matrices $R_x(\theta)$ and $R_y(\theta)$ for rotation about the x and y axes respectively.

3.5 HOMOGENEOUS COORDINATES

Rotation is a linear operation, but translation is not; it cannot be expressed as a matrix multiplication in ordinary 2D or 3D coordinates. Homogeneous coordinates resolve this by appending an extra coordinate equal to 1:

$$\mathbf{p}_h = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (3.5)$$

This allows both rotation and translation to be applied through a single matrix multiplication.

3.6 TRANSFORMATION MATRICES

The combined rotation-translation transformation matrix in homogeneous form is:

$$T(\theta, t_x, t_y) = \begin{bmatrix} \cos \theta & -\sin \theta & t_x \\ \sin \theta & \cos \theta & t_y \\ 0 & 0 & 1 \end{bmatrix} \quad (3.6)$$

Applying this to a homogeneous position vector produces the rover's new position and orientation in a single step:

$$\mathbf{p}'_h = T(\theta, t_x, t_y) \mathbf{p}_h \quad (3.7)$$

3.7 MATRIX MULTIPLICATION AND TRANSFORMATION CHAINING

Each command is expressed as a transformation matrix in the rover's *local* frame at the moment it is issued (Section 3.2). When the rover executes a sequence of n commands starting from an initial world transform W_0 , the overall world transform after all n commands is built by right-multiplying each local transform onto the accumulated result, in chronological order:

$$W_n = W_0 \cdot T_1 \cdot T_2 \cdots T_n \quad (3.8)$$

Because each T_i is defined relative to the rover's frame at step i , right-multiplication (rather than left-multiplication) correctly folds a local instruction into the accumulated world-frame pose. This single composite matrix is mathematically guaranteed to yield the same final position and heading as applying each T_i step by step, since matrix multiplication is associative.

CRITICAL DETAIL

Matrix multiplication is not commutative: $T_1 \cdot T_2 \neq T_2 \cdot T_1$ in general. The order in which transformations are chained directly affects the rover's final position and heading. This is one of the most common sources of implementation error in this project.

3.8 CONFIGURABLE INITIAL WORLD TRANSFORM

W_0 is not fixed at the identity matrix. It is built directly from a user-supplied starting pose (x_0, y_0, θ_0) :

$$W_0 = T(\theta_0, x_0, y_0) \quad (3.9)$$

using the combined transformation matrix from Section 3.7. This is what makes the rover's final resting position and heading fully manually configurable rather than hardcoded: changing (x_0, y_0, θ_0) or command sequence changes every subsequent W_i , and therefore the end point, without altering any code.

3.9 DERIVING A ROUTE BETWEEN TWO POINTS

Given only a start pose (x_0, y_0, θ_0) and a target end point (x_1, y_1) , the simplest command sequence this rover's model can express is a single rotation followed by a single forward translation: turn to face the target, then drive straight to it. This section derives that sequence directly from the displacement geometry.

The displacement vector from start to end is:

$$\Delta x = x_1 - x_0, \quad \Delta y = y_1 - y_0 \quad (3.10)$$

The straight-line distance to the target follows from the Pythagorean theorem:

$$d = \sqrt{\Delta x^2 + \Delta y^2} \quad (3.11)$$

The heading required to face the target is derived via $\text{atan2}(\Delta y, \Delta x)$, which works in all quadrants unlike $\arctan(\Delta y / \Delta x)$:

$$\theta_{\text{target}} = \text{atan2}(\Delta y, \Delta x) \quad (3.12)$$

The rotation the rover must execute before driving is the difference between this required heading and its current heading, normalized to a signed angle in $[-180^\circ, 180^\circ)$ so it always represents the shorter turn:

$$\Delta\theta_{\text{initial}} = \text{norm}(\theta_{\text{target}} - \theta_0) \quad (3.13)$$

If a specific final heading θ_1 is also requested, a second rotation is appended after the drive:

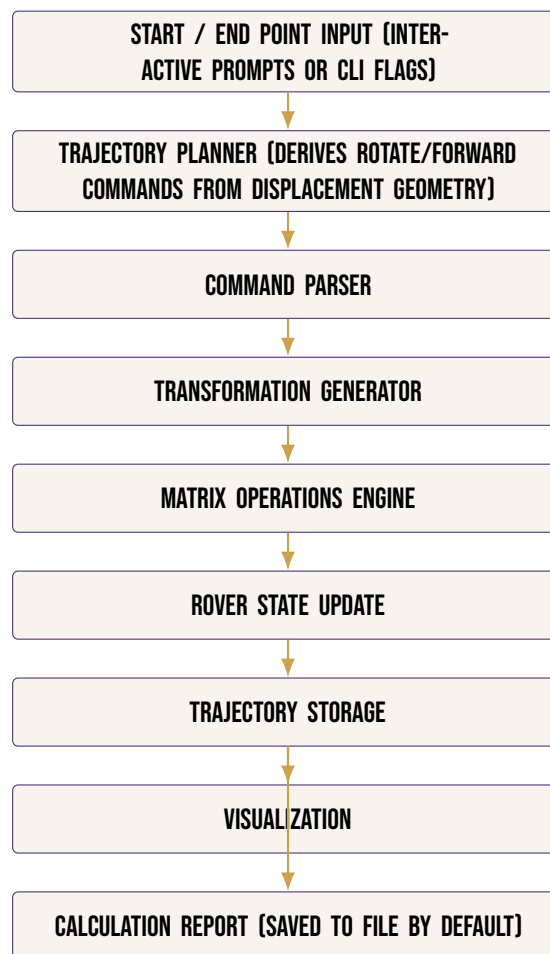
$$\Delta\theta_{\text{final}} = \text{norm}(\theta_1 - \theta_{\text{target}}) \quad (3.14)$$

WHY ROTATE-THEN-DRIVE, NOT A CURVED PATH

This rover's command set only supports pure rotation and pure straight-line translation (Sections 3.4 and 3.3); it has no primitive for moving along a curve. Rotate-then-drive is therefore the minimal-command path connecting any two poses this model can express, analogous to the straight-line segments of a simplified Dubins path.

PROPOSED SYSTEM ARCHITECTURE

4.1 PIPELINE OVERVIEW



TWO INPUT MODES

The Command Parser accepts commands from either source: the Trajectory Planner (default point-to-point mode, shown above) or a directly supplied command sequence (legacy explicit mode, via `--commands/--commands-file`), which bypasses the Trajectory Planner entirely.

4.2 COMPONENT DESCRIPTIONS

- **Start / End Point Input:** The rover's starting pose (x_0, y_0, θ_0) and target end point (x_1, y_1, θ_1) , collected via interactive console prompts (the default) or CLI flags (`--start-x/--end-x`, etc.). Nothing about the mission or its resulting end point is hardcoded.
- **Trajectory Planner:** Computes the minimal rotate-drive command sequence from displacement geometry (Section 3.10).
- **Command Parser:** Converts raw command strings into typed instructions consumed by the pipeline (planner or explicit mode).
- **Transformation Generator:** Converts each structured command into the corresponding transformation matrix (rotation, translation, or both).
- **Matrix Operations Engine:** Performs matrix multiplication, composes chained transformations, and applies them to the rover's homogeneous position vector. Every matrix built and multiplication performed is retained as a calculation record for step-by-step arithmetic review.
- **Rover State Update:** Updates the rover's stored position and heading based on the output of the matrix operations engine.
- **Trajectory Storage:** Appends each new state to a running history so the full path can be reconstructed later.
- **Visualization:** Renders the stored trajectory as a 2D (or 3D) plot showing the rover's path and orientation over time.
- **Calculation Report:** Formats retained calculation records into a step-by-step numeric report (local matrix formulas, dot products, state extraction formulas), saved by default to `matrix_operations.txt` for instructor review.

STEP-BY-STEP DEVELOPMENT PLAN

PHASE 1: PROJECT SETUP AND TECHNOLOGY SELECTION

Goal: Establish the development environment and finalize the technology stack.

Mathematical concepts: None (planning phase).

Programming tasks: Initialize repository; install chosen libraries; set up plotting environment; define project folder structure.

Expected output: A working, empty project skeleton that compiles/runs and can render a blank plot.

Common mistakes: Choosing a visualization library before confirming it supports the required 2D/3D plotting; not setting up version control from the start.

PHASE 2: IMPLEMENT VECTOR REPRESENTATION

Goal: Represent the rover's state as a vector and implement basic vector operations.

Mathematical concepts: Vectors, vector addition, position representation.

Programming tasks: Define a rover state class/struct holding x, y, θ ; implement getters/setters; implement a print/debug method.

Expected output: A rover object that can be instantiated with an initial state and printed.

Common mistakes: Storing angle in degrees in one part of the code and radians in another without conversion.

PHASE 3: IMPLEMENT MOVEMENT AND TRANSLATION

Goal: Apply pure translation to update rover position.

Mathematical concepts: Translation vectors, vector addition.

Programming tasks: Implement a `translate(tx, ty)` function; ensure translation is applied in the correct frame (local vs. world).

Expected output: Rover position updates correctly for forward, backward, or lateral movement commands.

Common mistakes: Applying translation in the world frame when the command was meant to be relative to the rover's current heading.

PHASE 4: IMPLEMENT ROTATION MATRICES

Goal: Apply rotation to update rover heading.

Mathematical concepts: 2D rotation matrices, trigonometric functions.

Programming tasks: Implement a `rotate(theta)` function constructing $R(\theta)$ and applying it to the heading/position as required.

Expected output: Rover heading updates correctly; test with 90° , 180° , 270° rotations.

Common mistakes: Mixing degrees and radians when calling `sin/cos`; sign errors producing rotation in the wrong direction.

PHASE 5: IMPLEMENT HOMOGENEOUS TRANSFORMATIONS

Goal: Unify rotation and translation into a single homogeneous transformation matrix.

Mathematical concepts: Homogeneous coordinates, combined transformation matrices.

Programming tasks: Implement a function that builds $T(\theta, t_x, t_y)$; convert rover position to homogeneous form before multiplication.

Expected output: A single matrix-vector multiplication correctly reproduces the combined effect of Phase 3 and Phase 4.

Common mistakes: Forgetting to append/strip the homogeneous coordinate consistently.

PHASE 6: IMPLEMENT TRANSFORMATION CHAINING

Goal: Support execution of an arbitrary sequence of commands via matrix composition.

Mathematical concepts: Matrix multiplication, non-commutativity, associativity.

Programming tasks: Implement a loop or reduction that multiplies transformation matrices in the correct order; apply the resulting composite matrix to the initial state.

Expected output: A sequence of five or more commands produces a final state matching manual calculation.

Common mistakes: Multiplying matrices in reverse order; not resetting the accumulator matrix between simulation runs.

PHASE 7: CREATE TRAJECTORY VISUALIZATION

Goal: Visualize the rover's full path and orientation history.

Mathematical concepts: Coordinate plotting, vector fields (for heading arrows).

Programming tasks: Store each intermediate state; plot x vs. y as a path; overlay heading arrows at each step.

Expected output: A 2D plot showing the rover's trajectory with directional markers.

Common mistakes: Only plotting the final position instead of the full stored history; incorrect axis scaling that distorts the path shape.

PHASE 8: TESTING AND DEBUGGING

Goal: Verify correctness of the full pipeline against known cases.

Mathematical concepts: Numerical verification, tolerance-based floating point comparison.

Programming tasks: Implement the test cases from Chapter 8; fix discrepancies between expected and computed output.

Expected output: All test cases pass within a defined numerical tolerance (e.g. 10^{-6}).

Common mistakes: Comparing floating point values with exact equality instead of a tolerance threshold.

PHASE 9: MATRIX CALCULATION REPORTING

Goal: Expose every matrix computation for instructor review, not just final numeric results.

Mathematical concepts: Explicit local matrix formulas per command type; element-by-element dot-product expansion of $W_n = W_{n-1} \cdot T_{\text{local}}$; composite verification against the step-by-step result.

Programming tasks: Log every local transform, previous world transform, and resulting world transform per command; build a report formatter that expands each of the 9 output matrix entries as an explicit sum-of-products, substitutes numeric $\cos \theta / \sin \theta$ values for rotations, and shows the state-extraction formulas applied to the final matrix; save the report to a text file by default and offer a console-print option.

Expected output: A saved report (`matrix_operations.txt`) showing every matrix used to reach the end point, with each multiplication broken into its 9 individual dot products and an explicit "matches step-by-step result: YES" confirmation.

Common mistakes: Computing the composite-matrix verification from the identity matrix regardless of the configured starting pose, which silently breaks the verification the moment a non-default start pose is used.

PHASE 10: POINT-TO-POINT ROUTE PLANNING

Goal: Replace manually scripted command sequences with a workflow that asks for a start point and an end point and derives the trajectory automatically.

Mathematical concepts: Displacement vector $(\Delta x, \Delta y)$; distance via

$\sqrt{\Delta x^2 + \Delta y^2}$; required heading via $\text{atan2}(\Delta y, \Delta x)$; signed angle normalization to $[-180^\circ, 180^\circ)$.

Programming tasks: Implement a planner that derives the minimal rotate-then-drive (optionally plus a final rotate) command sequence from a start pose and target point; build interactive console prompts that collect the start and end pose with input validation, sensible defaults, and a compass-direction summary; support the same start/end configuration via CLI flags for non-interactive use.

Expected output: Running the program with no arguments prompts for a start and end point, prints a full geometric derivation of the resulting commands, then runs and reports on them exactly as any other mission.

Common mistakes: Forgetting to skip the initial rotation when the start heading already points at the target (producing a spurious near-zero ROTATE command); not handling a start point that coincides with the end point.

SUGGESTED IMPLEMENTATION OPTIONS

6.1 COMPARISON OF OPTIONS

Option	Strengths	Weaknesses
Python (NumPy + Matplotlib)	Concise matrix syntax; built-in linear algebra; fast plotting; large community support	Slower raw execution than compiled languages (not relevant at this project's scale)
C++ (Eigen + OpenGL/SFML)	High performance; industry-relevant for robotics; strong for real-time rendering	Longer development time; steeper learning curve for a course project
MATLAB	Native matrix syntax; integrated plotting; widely taught in engineering courses	Licensing dependency; less portable outside academic environments

6.2 RECOMMENDATION

RECOMMENDED: PYTHON WITH NUMPY AND MATPLOTLIB

NumPy provides direct, readable support for matrix construction and multiplication (`@` operator or `numpy.matmul`), which maps closely onto the mathematical notation used in Chapter 3. Matplotlib provides sufficient 2D (and basic 3D) plotting for trajectory visualization without additional setup overhead. For a Linear Algebra course project where the priority is demonstrating mathematical correctness rather than building a production robotics system, Python minimizes implementation friction and maximizes time available for verifying the mathematics.

ALGORITHM DESIGN

Rover Initialization

```

FUNCTION initialize_rover(x0, y0, theta0):
    // x0, y0, theta0 are user-supplied -- never hardcoded
    W0 <- build_transformation_matrix(theta0, x0, y0)
    rover.world_transform <- W0
    rover.position        <- (x0, y0)
    rover.heading         <- theta0
    trajectory            <- [ (x0, y0, theta0) ]
    calculation_log       <- [ ]
    RETURN rover, trajectory, calculation_log

```

Deriving a Route from a Start and End Point

```

FUNCTION plan_route_to_point(x0, y0, theta0, x1, y1,
                             theta1_optional):

    dx <- x1 - x0
    dy <- y1 - y0
    distance <- sqrt(dx^2 + dy^2)

    commands <- [ ]
    IF distance > TOLERANCE:
        heading_to_target <- degrees(atan2(dy, dx))
        initial_turn <- normalize_degrees(heading_to_target
                                           - theta0)

        IF abs(initial_turn) > TOLERANCE:
            commands.append( ROTATE(initial_turn) )
        commands.append( FORWARD(distance) )
        heading_after_drive <- heading_to_target
    ELSE:
        heading_after_drive <- theta0 // start/end coincide

    IF theta1_optional IS GIVEN:
        final_turn <- normalize_degrees(theta1_optional
                                         - heading_after_drive)

        IF abs(final_turn) > TOLERANCE:
            commands.append( ROTATE(final_turn) )

    RETURN commands // minimal rotate-then-drive route

```

Angle Normalization

```
FUNCTION normalize_degrees(angle):  
    // Wraps any angle to [-180, 180) for shortest turn  
    RETURN ((angle + 180) MOD 360) - 180
```

Building a Command's Local Transform

```
FUNCTION local_transform_for_command(command):  
    IF command.type == FORWARD:  
        RETURN build_translation_matrix(command.value, 0)  
    IF command.type == BACKWARD:  
        RETURN build_translation_matrix(-command.value, 0)  
    IF command.type == ROTATE:  
        RETURN build_rotation_matrix(radians(command.value))
```

Updating Rover Position

```
FUNCTION update_state(rover, command):  
    W_previous <- rover.world_transform  
    T_local    <- local_transform_for_command(command)  
    W_new      <- W_previous * T_local // right-mult  
    rover.world_transform <- W_new  
    rover.position, rover.heading <- extract_state(W_new)  
    RETURN rover, W_previous, T_local, W_new
```

Extracting State From a World Transform

```
FUNCTION extract_state(W):  
    x      <- W[0][2]  
    y      <- W[1][2]  
    heading <- atan2(W[1][0], W[0][0])  
    RETURN (x, y, heading)
```

Storing Trajectory

```
FUNCTION record_state(trajjectory, rover):  
    trajectory.append( (rover.x, rover.y, rover.heading) )  
    RETURN trajectory
```

Recording a Calculation Step

```
FUNCTION record_calculation(log, command, W_previous,  
                           T_local, W_new):  
    // Retain all matrices involved in update for review  
    step <- { command, W_previous, T_local, W_new }  
    log.append(step)  
    RETURN log
```

Generating the Calculation Report

```

FUNCTION generate_calculation_report(trajectory,
                                     calculation_log, W0):
    PRINT "Initial world transform W0:", W0

    FOR each step IN calculation_log:
        PRINT "Command parameter (distance d, or angle theta"
        PRINT " substituted numerically)"
        PRINT "Local matrix formula, e.g. T_local = "
        PRINT " [[cos(theta),-sin(theta),0],\"
        PRINT "  [sin(theta),cos(theta),0],[0,0,1]]"
        PRINT "T_local:" , step.T_local
        PRINT "Previous world transform W_(n-1):", step.W_previous

        // Expand each entry as explicit dot product
        FOR i IN 0..2:
            FOR j IN 0..2:
                PRINT "W_n[i,j] = SUM_k( W_(n-1)[i,k] *\"
                PRINT "      T_local[k,j] ) = \"
                PRINT "      \" + step.W_new[i][j]

        PRINT "Resulting world transform W_n:", step.W_new
        PRINT "State extraction: x = W_n[0,2], y = W_n[1,2],\"
        PRINT "heading = atan2(W_n[1,0], W_n[0,0])\"

    composite <- W0
    FOR each step IN calculation_log:
        composite <- composite * step.T_local
    PRINT "Composite matrix (all steps combined):\", composite
    PRINT "Matches step-by-step final state:\",
        (extract_state(composite) == trajectory.last)

    // Saved to a text file by default (matrix_operations.txt)
    IF NOT show_on_console:
        write_to_file(report_text, \"matrix_operations.txt\")
    ELSE:
        PRINT report_text

```

Displaying the Path

```

FUNCTION plot_trajectory(trajectory):
    xs <- extract_x(trajectory)
    ys <- extract_y(trajectory)
    plot(xs, ys)
    FOR each state IN trajectory:
        draw_heading_arrow(state.x, state.y, state.theta)

```

```
show_plot()
```

TESTING STRATEGY

8.1 TEST CASE CATEGORIES

Case	Description	Expected Result
Straight movement	Forward command only, no rotation	Position changes along heading direction; heading unchanged
Rotation only	Rotate command only, no translation	Heading changes; position unchanged
Combined	One rotation followed by one translation	Position changes along the <i>new</i> heading direction
Chained	Five or more mixed commands in sequence	Final state matches manual matrix-multiplication calculation
Boundary	Zero-distance movement; $360^\circ/0^\circ$ rotation; very large distance values	State remains numerically stable and mathematically consistent
Manual configuration	Non-default starting pose combined with an arbitrary command program	Resulting end point reflects the configured pose and commands, not a hardcoded default
Calculation log integrity	Composite matrix built from the calculation log, starting from the configured W_0	Matches the step-by-step final state exactly, for both default and non-default starting poses
Route planning	Start/end pairs requiring zero, one, or two rotations; a requested end heading; coincident start and end points	Derived command sequence reaches the target position (and heading, if requested) to within tolerance
Interactive prompts	Blank input (defaults/skip), invalid input (re-prompt), and a full valid input sequence	Correct value returned or re-prompted in every case; no crash on malformed input

8.2 TESTING CHECKLIST

- ☐ Verify straight-line movement test case.
- ☐ Verify rotation-only test case.
- ☐ Verify combined rotation-translation test case.
- ☐ Verify multi-step chained transformation test case against manual calculation.
- ☐ Verify boundary cases (zero movement, full rotation, large values).
- ☐ Verify a non-default starting pose produces a correctly shifted end point.
- ☐ Verify the composite-matrix calculation log check passes for both default and non-default starting poses.
- ☐ Verify the route planner produces the correct turn-drive-turn command sequence for sample start/end pairs.
- ☐ Verify interactive prompts handle defaults, blank/optional input, and invalid input without crashing.
- ☐ Confirm numerical results match expected values within tolerance 10^{-6} .
- ☐ Confirm trajectory plot visually matches expected path shape.

CURRENT AUTOMATED TEST COUNT

The full suite (`python3 rover_navigation_simulator.py --run-tests`) runs 44 automated tests across all categories above and must show OK before submission.

PROJECT MANAGEMENT CHECKLIST

9.1 RESEARCH

- ☐ Understand mathematical concepts (vectors, matrices, and homogeneous coordinates).
- ☐ Finalize technology stack.
- ☐ Study transformation matrices in depth.

9.2 IMPLEMENTATION

- ☐ Create rover model.
- ☐ Implement vectors.
- ☐ Implement transformations.
- ☐ Implement matrix multiplication.
- ☐ Implement visualization.
- ☐ Implement CLI configuration for starting pose and command program.
- ☐ Implement matrix calculation logging and report generation.
- ☐ Implement the route planner (start/end point → movement commands).
- ☐ Implement interactive start/end point prompts with input validation.

9.3 DOCUMENTATION

- ☐ Prepare explanation of approach and design decisions.
- ☐ Add mathematical derivations.
- ☐ Add screenshots of simulation output.
- ☐ Prepare presentation slides.

9.4 FINAL REVIEW

- ☐ Code cleanup.
- ☐ Verify mathematical correctness.
- ☐ Practice demonstration.

CHAPTER 10

VIVA PREPARATION

10.1 VIVA READINESS CHECKLIST

- ☐ Able to explain every mathematical concept without notes.
- ☐ Able to trace through one full example by hand.
- ☐ Able to justify every implementation decision.
- ☐ Able to explain limitations and possible improvements honestly.
- ☐ Rehearsed a live demonstration at least twice.

10.2 MATHEMATICS QUESTIONS

Q: What is a vector?

A: An ordered list of numbers representing a quantity with both magnitude and direction; here it represents the rover's position or state.

Q: Why are matrices used for transformations?

A: A matrix acts as a linear operator that maps an input vector to a transformed output vector, allowing rotation and translation to be expressed and computed systematically.

Q: Explain rotation matrices.

A: A rotation matrix $R(\theta)$ rotates a vector by angle θ about the origin while preserving its length; it is built from sine and cosine of the rotation angle.

Q: Why do transformation matrices need multiplication?

A: Because applying several transformations in sequence corresponds mathematically to multiplying their matrices together, producing one composite matrix that reproduces the full sequence in a single operation.

Q: Explain homogeneous coordinates.

A: An extra coordinate (typically 1) is appended to a position vector so that translation, which is not linear in ordinary coordinates, can be expressed as a matrix multiplication alongside rotation.

Q: Difference between translation and rotation.

A: Translation shifts a point's position without changing orientation; rotation changes orientation about a fixed point without necessarily changing distance from that point.

Q: Explain matrix multiplication order.

A: Matrix multiplication is not commutative; transformations must be composed in the exact order intended. Reversing the order produces a different final result.

10.3 IMPLEMENTATION QUESTIONS

Q: How does the simulator process commands?

A: In the default mode, a start and end point are turned into commands by the route planner; in the legacy mode, commands are supplied directly. Either way, each command is parsed into a structured instruction, converted into a transformation matrix, then applied in sequence to update the rover's state.

Q: How is the rover position stored?

A: As a state vector (or homogeneous vector) containing x , y , and heading θ , updated after each command and appended to a trajectory history.

Q: How are transformations chained?

A: By multiplying the individual transformation matrices together in the order the commands were issued, then applying the resulting composite matrix to the initial state.

Q: How is the trajectory generated?

A: By recording the rover's state after every command and later plotting the full sequence of recorded positions.

Q: Why was a specific programming language chosen?

A: Based on the tradeoffs in Chapter 6; Python was recommended for its concise matrix syntax and integrated plotting, minimizing implementation overhead for a mathematically-focused project.

Q: How is the rover's end point made manually configurable?

A: The user supplies a start pose and a target end point (interactively or via CLI flags); the route planner derives the command sequence from their geometry, so the initial world transform W_0 and every subsequent transform depend entirely on user input, not hardcoded values.

Q: How does the simulator turn a start and end point into commands?

A: It computes the displacement vector between them, derives the distance and required heading via atan2 , rotates to face the target, drives straight to it, and optionally rotates again to reach a requested final heading (Section 3.10).

Q: How are the matrix calculations shown for review?

A: Every command execution retains the local, previous world, and resulting world transforms. These are formatted into a step-by-step report with expanded dot products, saved to a text file by default, and cross-checked against composite matrix results.

10.4 PROJECT UNDERSTANDING QUESTIONS

Q: Explain the complete pipeline.

A: A start and end point are turned into commands by the route planner (or supplied directly in legacy mode); commands are parsed, converted into transformation matrices, composed via matrix multiplication, applied to update the rover's state, recorded into a trajectory, and finally visualized and reported on.

Q: Explain a sample rover movement step-by-step.

A: State the initial position and heading, show the transformation matrix for the given command, perform the matrix-vector multiplication explicitly, and state the resulting new position and heading.

Q: Explain limitations and possible improvements.

A: The current model may assume ideal, noise-free movement; possible improvements include adding sensor noise models, obstacle detection, 3D terrain, or a more sophisticated path-planning layer.

PRESENTATION PREPARATION

11.1 RECOMMENDED 15-MINUTE STRUCTURE

#	Slide	Time	Key Content
1	Introduction	1 min	Project name, team, one-line summary
2	Problem Statement	1 min	What the simulator solves and why
3	Mathematical Concepts	2.5 min	Vectors, rotation matrices, homogeneous coordinates
4	System Architecture	1.5 min	Pipeline diagram from Chapter 4
5	Implementation	2 min	Language choice, key modules
6	Demonstration	3 min	Live or recorded run of the simulator
7	Results	1.5 min	Sample trajectories, correctness checks
8	Challenges	1 min	Key obstacles encountered
9	Future Improvements	1 min	Extensions such as 3D or obstacle avoidance
10	Conclusion	0.5 min	Summary and closing statement

PRESENTATION REMINDERS

Slides should minimize on-screen text and favor visuals: diagrams, plotted trajectories, and pseudocode rather than dense paragraphs. Slides are to be finalized and submitted immediately following the presentation.

CHAPTER 12

FINAL PROJECT COMPLETION CHECKLIST

- ☐ All ten development phases completed and verified.
- ☐ All test cases from Chapter 8 passing.
- ☐ Trajectory visualization functioning correctly for chained commands.
- ☐ Mathematical derivations documented and cross-checked.
- ☐ Manual configuration of starting pose and command program verified end-to-end.
- ☐ Matrix calculation report generated and reviewed for at least one non-default starting pose.
- ☐ Presentation slides finalized per the structure in Chapter 11.
- ☐ Viva questions in Chapter 10 rehearsed by all group members.
- ☐ Code cleaned, commented, and organized for submission.
- ☐ Final report/documentation assembled and proofread.
- ☐ Submission files verified against course submission requirements.